RASU - Premium Futuristic Fashion E-commerce Platform

RASU is a full-stack e-commerce project built for a modern fashion shopping experience. It combines a visually rich React frontend with a robust Node.js and MongoDB backend, including authentication, order management, support, and AI-powered chat assistance.

Project Overview

This project was designed to deliver:

• A premium, responsive shopping interface

• Secure user authentication with OTP verification

• Cart, wishlist, checkout, and order tracking workflows

• Profile management with image upload support

• Support system with smart AI chat behavior

RASU is optimized to run across mobile, tablet, and desktop devices with a strong focus on usability, performance, and maintainability.

Tech Stack

Frontend

• React (TypeScript)

• Vite

• Tailwind CSS

• Framer Motion

• React Router

• React Query

• Radix UI + shadcn-style component patterns

Backend

• Node.js

• Express

• MongoDB + Mongoose

• JWT Authentication

• bcrypt

• Nodemailer (email OTP/reset)

• Multer (profile image upload)

Optional Integrations

• Twilio (phone OTP)

• Razorpay (payments)

Core Features

1. Authentication and Account Security

• User signup and login

• Email OTP verification

• Optional phone OTP support

• Forgot-password and reset-password flow

• JWT-based protected routes

2. Product and Shopping Experience

• Dynamic product browsing and filtering

• Category support (men, women, accessories, trending)

• Wishlist and cart workflows

• Smooth UI interactions with motion effects

3. Checkout and Orders

• Shipping form with validations

• Order creation via backend API

• Order history display in profile

• Order tracking entry points

4. Profile Management

• Editable personal details and address

• Profile image upload and delete

• Account status and verification indicators

• Clickable sections for wishlist, cart, and orders

5. Support and AI Chat

• Support message API with MongoDB persistence

• Live chat widget on frontend

• Context-aware support responses

• Session memory for short user preferences in chat

Folder Structure (High-Level)

• `frontend/` - React + Tailwind application

• `backend/` - Express API and MongoDB models/routes

• `Docs/` - project-related documentation assets

• `ai-service/` - auxiliary AI-related services (if used)

API Highlights

Auth

• `POST /api/auth/register`

• `POST /api/auth/login`

• `POST /api/auth/verify-otp`

• `POST /api/auth/forgot-password`

• `POST /api/auth/reset-password`

Users

• `GET /api/users/profile`

• `PUT /api/users/profile`

• `POST /api/users/upload`

• `DELETE /api/users/upload`

Orders

• `POST /api/orders`

• `GET /api/orders/user/:userId`

• `GET /api/orders/:orderId`

Support

• `POST /api/support/messages`

• `POST /api/support/chat`

Environment Setup

Backend (`backend/.env`)

Required:

• `PORT`

• `MONGO_URI`

• `JWT_SECRET`

• `CLIENT_URL`
• `SMTP_HOST`
• `SMTP_PORT`
• `SMTP_SECURE`
• `SMTP_USER`
• `SMTP_PASS`
• `SMTP_FROM`

Optional:

• `RAZORPAY_KEY_ID`
• `RAZORPAY_KEY_SECRET`
• `TWILIO_ACCOUNT_SID`
• `TWILIO_AUTH_TOKEN`
• `TWILIO_PHONE_NUMBER`

Running the Project

Backend

1. Open terminal in `backend/`
2. Install dependencies: `npm install`
3. Start server: `npm run dev`

Frontend

1. Open terminal in `frontend/`
2. Install dependencies: `npm install`
3. Start app: `npm run dev`

Production Readiness Notes

The project includes several production-focused improvements:

• Responsive design with mobile-first Tailwind patterns
• Route-level lazy loading and fallback UI
• Centralized frontend API helper for cleaner fetch logic
• Modular backend service extraction for maintainability
• Better UI touch targets and accessibility for key controls

Suggested Next Steps

• Add admin dashboard for support messages and order operations

• Add CI pipeline (lint, type-check, build)

• Integrate cloud media storage for profile images

• Add unit and integration tests for critical flows

• Harden rate limiting and request validation policies

Author Note

This README reflects the current implementation state of the RASU project and is written for practical team onboarding, technical review, and project handover use.